

Lab Report No 8
Implementation of Color Models and Color Image
Processing

Digital Image Processing



Submitted By:

[Huzaifa Shahbaz]

[21-CE-009]

[Aqsa Shah]

[21-CE-039]

Submitted to:

Engr. Hareem Khan

Dated:

29th Nov, 2024

Department of Computer Engineering,
HITEC University, Taxila

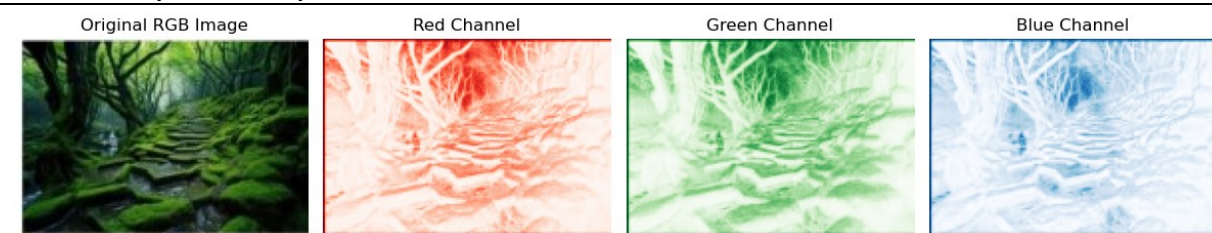
Lab Example No 1:

Solution:

Brief description (3-5 lines)
This is an additive model, i.e. the colours present in the light add to form new colours, and is appropriate for the mixing of coloured light for example. The image shows the additive mixing of red, green and blue primaries to form the three secondary colours yellow (red + green), cyan (blue + green) and magenta (red + blue), and white ((red + green + blue).
The code
<pre>import cv2 import matplotlib.pyplot as plt # Load the image image = cv2.imread('input_image.jpg') image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB) # Convert to RGB for display # Split the image into R, G, B channels r_channel, g_channel, b_channel = cv2.split(image_rgb) # Display the original image and each channel plt.figure(figsize=(12, 6)) # Original Image plt.subplot(1, 4, 1) plt.imshow(image_rgb) plt.title("Original RGB Image") plt.axis('off') # Red Channel plt.subplot(1, 4, 2) plt.imshow(r_channel, cmap='Reds') plt.title("Red Channel") plt.axis('off') # Green Channel plt.subplot(1, 4, 3) plt.imshow(g_channel, cmap='Greens') plt.title("Green Channel") plt.axis('off')</pre>

```
# Blue Channel
plt.subplot(1, 4, 4)
plt.imshow(b_channel, cmap='Blues')
plt.title("Blue Channel")
plt.axis('off')
plt.tight_layout()
plt.show()
```

The results (Screenshot)



Lab Example No 2:

Solution:

Brief description (3-5 lines)

The CMY (cyan-magenta-yellow) model is a subtractive model appropriate to absorption of colours, for example due to pigments in paints. Whereas the RGB model asks what is added to black to get a particular colour, the CMY model asks what is subtracted from white. In this case, the primaries are cyan, magenta and yellow, with red, green and blue as secondary colours

The code

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
# Read the RGB image
I = cv2.imread('The CMY Model.jpeg')
I = cv2.cvtColor(I, cv2.COLOR_BGR2RGB) # Convert BGR to RGB
for proper color display
# Display the original RGB image
plt.figure()
plt.imshow(I)
```

```
plt.title("Original RGB Image")
plt.axis('off')
# Create a CMY image by subtracting each channel from 255
CMY = np.zeros_like(I) # Initialize CMY array with the same shape as I
R = I[:, :, 0] # Red channel
G = I[:, :, 1] # Green channel
B = I[:, :, 2] # Blue channel
# Compute CMY channels
CMY[:, :, 0] = 255 - R # Cyan (complement of red)
CMY[:, :, 1] = 255 - G # Magenta (complement of green)
CMY[:, :, 2] = 255 - B # Yellow (complement of blue)
# Display the CMY image
plt.figure()
plt.imshow(CMY)
plt.title("CMY Image")
plt.axis('off')
plt.show()
```

The results (Screenshot)

Original RGB Image



CMY Image



Lab Example No 2:

Solution:**Brief description (3-5 lines)**

As mentioned above, colour may be specified by the three quantities hue, saturation and intensity. This is the HSI model, and the entire space of colours that may be specified in this way showing the HSI solid on the left, and the HSI triangle on the right, formed by taking a horizontal slice through the HSI solid at a particular intensity. Hue is measured from red, and saturation is given by distance from the axis. Colours on the surface of the solid are fully saturated, i.e. pure colours, and the greyscale spectrum is on the axis of the solid. For these colours, hue is undefined.

The code

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
# Read the RGB image and normalize it to [0, 1]
Img = cv2.imread('import cv2')
import numpy as np
import matplotlib.pyplot as plt
# Read the RGB image and normalize it to [0, 1]
Img = cv2.imread('The HSI Model.jpg')
Img = cv2.cvtColor(Img, cv2.COLOR_BGR2RGB) # Convert BGR to RGB
Img = Img.astype(np.float32) / 255.0 # Normalize to range [0, 1]
# Display the original image
plt.figure()
plt.imshow(Img)
plt.title("Original RGB Image")
plt.axis('off')
Img = cv2.cvtColor(Img, cv2.COLOR_BGR2RGB) # Convert BGR to RGB
Img = Img.astype(np.float32) / 255.0 # Normalize to range [0, 1]
# Display the original image
plt.figure()
plt.imshow(Img)
plt.title("Original RGB Image")
plt.axis('off')
```

```

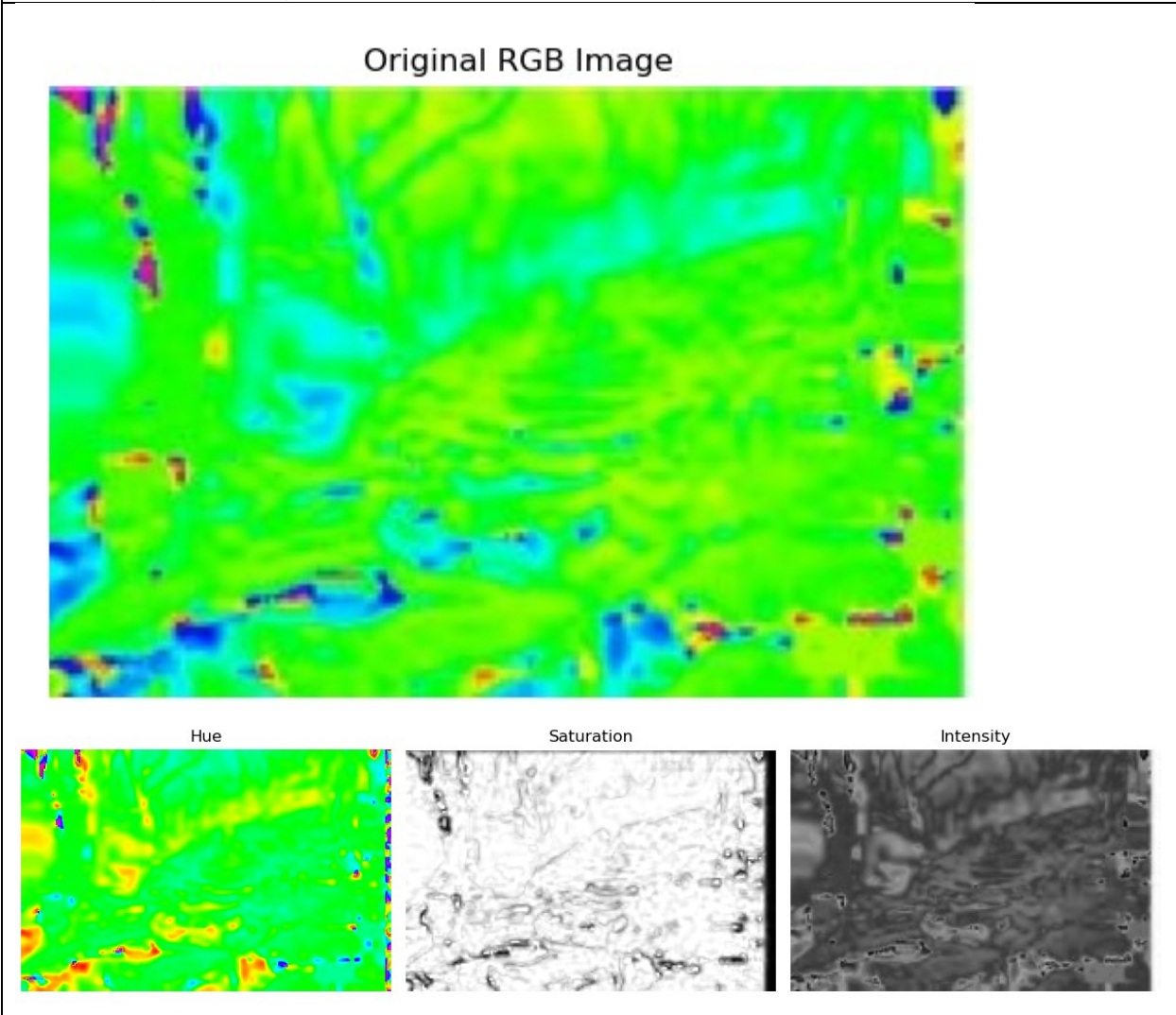
# Extract R, G, B channels
R = Img[:, :, 0]
G = Img[:, :, 1]
B = Img[:, :, 2]
# Hue calculation
numerator = 0.5 * (R - G + R - B)
denominator = np.sqrt((R - G) ** 2 + (R - B) * (G - B)) + 1e-7 #
Avoid division by zero
H = np.arccos(numerator / denominator)
# Adjust H where B > G
H[B > G] = 2 * np.pi - H[B > G]
H = H / (2 * np.pi) # Normalize to range [0, 1]
# Intensity calculation
I = (R + G + B) / 3
# Saturation calculation
minRGB = np.minimum(np.minimum(R, G), B)
S = 1 - 3 * minRGB / (R + G + B + 1e-7) # Avoid division by zero
# Combine H, S, and I into one visualization
HSI = np.stack((H, S, I), axis=-1)
# Display H, S, and I components
plt.figure(figsize=(12, 4))
# Hue
plt.subplot(1, 3, 1)
plt.imshow(H, cmap='hsv')
plt.title("Hue")
plt.axis('off')
# Saturation
plt.subplot(1, 3, 2)
plt.imshow(S, cmap='gray')
plt.title("Saturation")
plt.axis('off')
# Intensity
plt.subplot(1, 3, 3)
plt.imshow(I, cmap='gray')
plt.title("Intensity")

```



```
plt.axis('off')
plt.tight_layout()
plt.show()
```

The results (Screenshot)



Lab Task No 1:

Solution:

Brief description (3-5 lines)
Convert CMY to RGB with key color 0.412 where the formula is $[R \ G \ B] = 255 * [1 - C \ 1 - M \ 1 - Y] * [1 - key \ 1 - key \ 1 - key]$
The code


```

import cv2
import numpy as np
import matplotlib.pyplot as plt
# Import the image
img = plt.imread("The CMY Model.jpg")
# Normalize the BGR image to range [0, 1]
bgr = img.astype(float) / 255.0
# Calculate CMYK channels
with np.errstate(invalid='ignore', divide='ignore'):
    K = 1 - np.max(bgr, axis=2)
    C = (1 - bgr[..., 2] - K) / (1 - K + 1e-10) # Add a small value to
    # avoid division by zero
    M = (1 - bgr[..., 1] - K) / (1 - K + 1e-10)
    Y = (1 - bgr[..., 0] - K) / (1 - K + 1e-10)
# Stack the channels and scale to [0, 255]
CMYK = (np.dstack((C, M, Y, K)) * 255).astype(np.uint8)
# Split CMYK channels
C, M, Y, K = cv2.split(CMYK)
# Save each CMYK channel as a separate image
cv2.imwrite('C_Channel.jpg', C)
cv2.imwrite('M_Channel.jpg', M)
cv2.imwrite('Y_Channel.jpg', Y)
cv2.imwrite('K_Channel.jpg', K)
# Function to convert CMY to RGB
def cmy_to_rgb(c, m, y, key):
    r = (1 - c) * (1 - key)
    g = (1 - m) * (1 - key)
    b = (1 - y) * (1 - key)
    return r, g, b
# Example usage of the CMY to RGB conversion function
c = 1 - 0.412
m = 1 - 0.412
y = 1 - 0.412
key = 0.412
r, g, b = cmy_to_rgb(c, m, y, key)

```

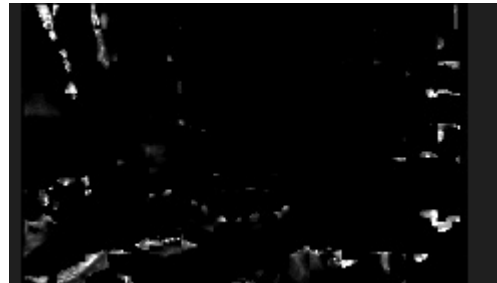
```
# Print the RGB values
print(f'R: {r:.3f}')
print(f'G: {g:.3f}')
print(f'B: {b:.3f}')
```

The results (Screenshot)

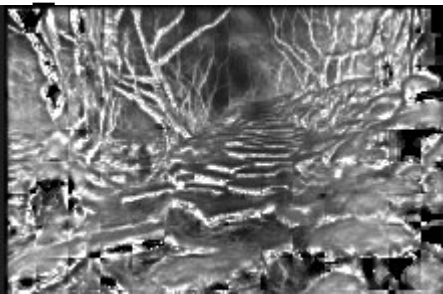
```
R: 0.242
G: 0.242
B: 0.242
```



K Channel



M Channel



Y Channel

Lab Task No 2:

Solution:

Brief description (3-5 lines)

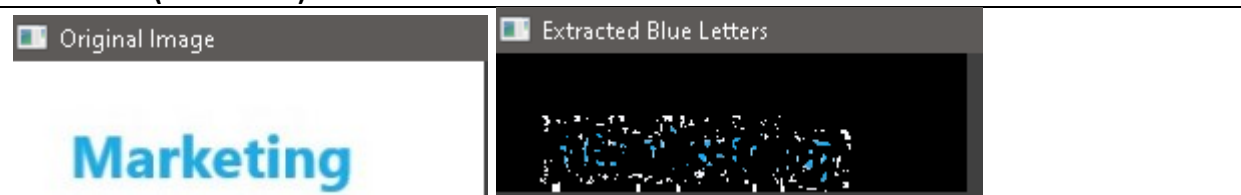
Extract blue color letters from the given image where the color range is (100,0,0) to (130,255,255)

The code

```
import cv2
import numpy as np
# Load the image
image = cv2.imread('marketing.jpg')
```

```
# Convert the image from BGR to HSV color space
hsv_image = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)
# Define the lower and upper bounds for the blue color in HSV
lower_blue = np.array([100, 0, 0])
upper_blue = np.array([130, 255, 255])
# Create a mask using the defined color range
mask = cv2.inRange(hsv_image, lower_blue, upper_blue)
# Bitwise-AND mask and original image to extract blue areas
extracted_blue = cv2.bitwise_and(image, image, mask=mask)
# Display the original image and the extracted blue areas
cv2.imshow('Original Image', image)
cv2.imshow('Extracted Blue Letters', extracted_blue)
# Wait for a key press and close the image windows
cv2.waitKey(0)
cv2.destroyAllWindows()
```

The results (Screenshot)



Lab Task No 3:

Solution:

Brief description (3-5 lines)

Apply HSV and RAINBOW color map to the gray scale image.

The code

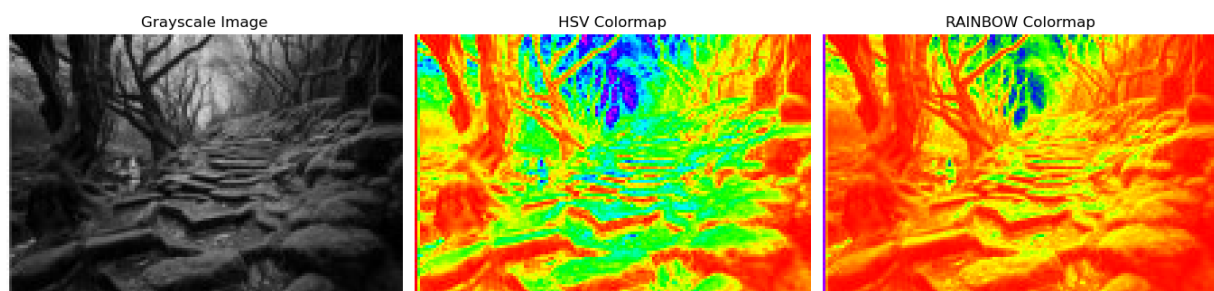
```
import cv2
import matplotlib.pyplot as plt
import numpy as np
# Load the grayscale image
image_path = 'grayscale.jpg' # Replace with the path to your image
```

```

grayscale_image = cv2.imread(image_path,
cv2.IMREAD_GRAYSCALE)
# Apply HSV and RAINBOW colormaps
hsv_colormap = cv2.applyColorMap(grayscale_image,
cv2.COLORMAP_HSV)
rainbow_colormap = cv2.applyColorMap(grayscale_image,
cv2.COLORMAP_RAINBOW)
# Display the results
plt.figure(figsize=(15, 5))
# Original grayscale image
plt.subplot(1, 3, 1)
plt.title("Grayscale Image")
plt.imshow(grayscale_image, cmap='gray')
plt.axis('off')
# HSV colormap
plt.subplot(1, 3, 2)
plt.title("HSV Colormap")
plt.imshow(cv2.cvtColor(hsv_colormap, cv2.COLOR_BGR2RGB))
plt.axis('off')
# RAINBOW colormap
plt.subplot(1, 3, 3)
plt.title("RAINBOW Colormap")
plt.imshow(cv2.cvtColor(rainbow_colormap,
cv2.COLOR_BGR2RGB))
plt.axis('off')
plt.tight_layout()
plt.show()

```

The results (Screenshot)



Conclusion

In conclusion, Color models and color image processing enhance the understanding and manipulation of images in different applications. Optimization of most specific image processing tasks could be achieved by using different color spaces like RGB, CMY, HSV, or YCbCr. Efficient image analysis and visualization occur when different techniques such as filtering, segmentation, or transformation are brought to the scene. These methods achieve the desired goal in computer vision, medical imaging, and multimedia. Ultimately, the use of color models contributes to image processing tasks by ensuring accuracy and flexibility in their handling.